

Client Performance in the Age of AI

A Lesson in Controlling Entropy

Jenna Zeigen

Node.JS Interactive @ RenderATL 2026

August 12, 2026

hi, i'm jenna

- Former Senior Staff perf girlie @ Notion & Slack
- Organizer @ NY UX Speed and Web Performance Meetup
- Find me on the internet @ @zeigenvector
- Find these slides @ jenna.is/at-renderatl



client performance has always
been a challenge.

how do we keep up?

why "do" performance?

- ~~So the graphs go in the right direction?~~
- ~~So we make more money??~~
- ~~So people don't write bad things about us on Hacker News???~~

✨ So our users have a ~~great pleasant~~
not-bad experience! ✨

the way we're working *and*
the things we're creating are
both different and we don't
have a great handle on it yet

we're mediated even farther
from the systems we're
building on top of

lower barrier to entry for
contributing to the
codebase

AI amplifies patterns,
both good and bad

we're a lot more
"productive" now

AI interfaces have brought
about different interaction
patterns

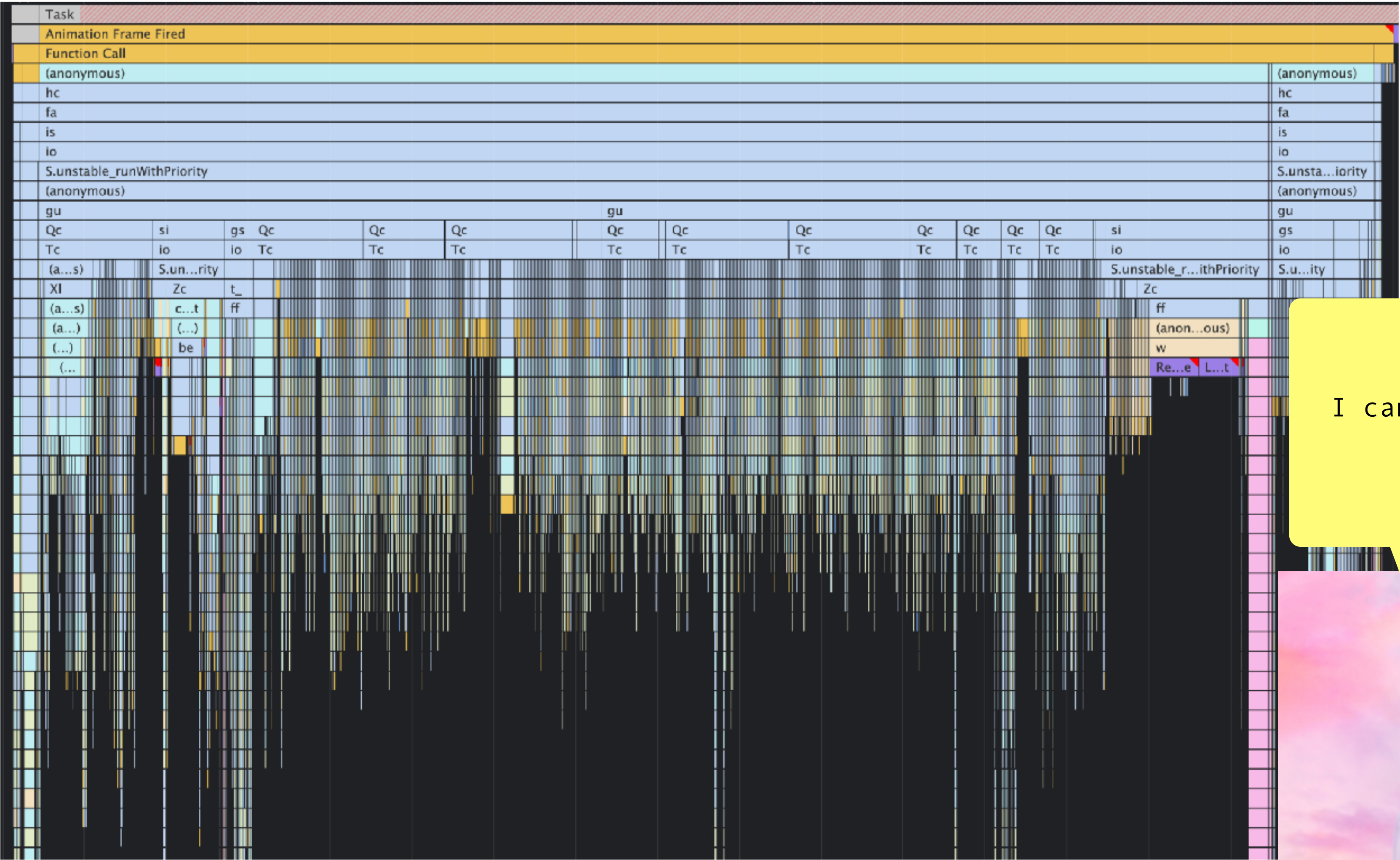
~*everything*~
is changing??

yet, "performance" hasn't
actually changed that much

performance issues are
often the result of
systemic issues

performance issues often
surface as discrete
antipatterns.

diagnosis: ~~murder~~ render



I can't pretend it's okay when it's not
It's death by a thousand cuts



so, what happens when

1. you understand the system less
2. (anti)patterns are amplified
3. you ship more code, faster than ever

OH NO



we're going up up up!!!!



so, how do we maintain good
experiences for our users
through all of this?

how do we control entropy??

so, what happens when

1. you understand the system less
2. (anti)patterns are amplified
3. you ship more code, faster than ever

tl;dr: use the same
playbook (mostly),
now with some AI help!

the oldest performance
wisdom in the book is
"always measure first"

oh... yes?



we're going up up up!!!!



what metrics make the
most sense for your
application?

you still need metrics

- INP = clicking + typing



jenna Oct 30th, 2025 at 3:33 PM

Not really an investigation, but... has there been any recommendations about INP for typing heavy applications? 200ms INP keystrokes seems like it's going to be a bad time for users, but canonical INP lore says <200ms is good. For context, our "clicky" INP p75 is 175ms but our keypress INP's is 40ms. Even curious what people's vibes on this are! (edited)



Barry Pollard Oct 30th, 2025 at 4:29 PM

200ms INP keystrokes seems like it's going to be a bad time for users, but canonical INP lore says <200ms is good.

Core Web Vitals targets are broad, high-level, and aimed to apply to all pages. They were also set with one eye to achievability: <https://web.dev/articles/defining-core-web-vitals-thresholds>.

I agree for typing, 200ms is often way too high and you should aim for lower. But, for simplicity we don't have different thresholds for typing and button clicks—which are two very different interactions.

Exactly what the target time should be, I don't know.

Less than 100ms is generally considered imperceptible to most people so maybe that's a good target? Though there's other research saying that's not a fixed threshold and people are different.

You could consider typing skills. Ideally you'd show each letter as it's typed. A good Words Per Minute (WPM) is 60, and advanced is 100. Let's take the 100 example cause even non-advanced typists can reach high speed for certain typing, even if they can't sustain it like an advanced typist could. A "word" is generally considered to be 5 letters for typing measurement so that means we need $100/60 = 1.667$ words per second or 8.3333 letters per second. Which means you need to respond to a letter being typed every $1000/8.3333 = 120$ milliseconds, if the rest of my calculations are correct.

does

"time to first token"
cover the whole
experience?

understanding your
system

fostering systemic "understanding"

- how do the frameworks and systems work?
- what's currently slow? why?
- can the AI help you better understand it?
- can you help the AI better understand it?
- documentation can be read by humans and robots!

creating
guardrails

employing AI skills

- create AI skills addressing known antipatterns that can be run by engineers seeking to optimize their code
- can you also run this skill in CI as a code review bot?

employing AI skills

- want something generic? you might be interested in Vercel's react-best-practices or million.js's react.doctor

performance is about
tradeoffs, but AI cannot be
taught "taste" or "it
depends" effectively

adding custom lint rules

- catch known antipatterns via lint rules with the help of AI!
- the most surefire way to get an AI to follow instructions is to rely on engineering best practices

adding custom lint rules

- "hey claude, can you help me write a lint rule that ensures the return value of useComputedStore is a stable value? and can you make it autofix if possible? thanks!"
- iterate until it works!

diagnosing
regressions

using chrome dev tools mcp

- "hey claude, can you load notion.com using Chrome Dev Tools MCP and click the AI button and profile how long it takes to open the chat popup?"

using chrome dev tools mcp

- "hey claude, can you load notion.com using Chrome Dev Tools MCP and click the AI button and profile how long it takes to open the chat popup?"
- honestly, this was disappointing!

using the profiler

- run a profile and ask the AI to find patterns in `*that*`
- "hey claude, I took this profile when clicking on the agent button. can you look at it and see what might be causing it to take the 600ms to open? thanks!"
- also works for the React Dev Tools profiler!

using the profiler

- "great! can you look at `Text.toHtml()` and tell me if you see any optimization opportunities?"
- AI is actually okay with vanilla JavaScript optimization!

speculative: heap snapshots

- this could also work for heap snapshots too



everything's so wrong
and I don't belong
living in your precious memory



you know i'd spend a hundred thousand tokens
if I could just see you!!



Summary ▾ Filter by class				
All objects ▾				
Constructor	Distance	Shallow Size		Retained Size ▾
▶ (compiled code) ×614,084	3	49,784 kB	12 %	123,531 kB 30 %
▶ (string) ×307,917	3	32,877 kB	8 %	106,300 kB 26 %
▶ system / ExternalStringDa...	4	73,428 kB	18 %	73,428 kB 18 %
▶ (array) ×818,031	3	41,194 kB	10 %	70,939 kB 17 %
▶ I ×83,228	11	7,324 kB	2 %	56,440 kB 14 %
▶ Map ×158,918	4	2,543 kB	1 %	45,314 kB 11 %
▶ Object ×158,888	3	4,237 kB	1 %	44,819 kB 11 %
▶ IK ×38,666	6	3,867 kB	1 %	41,704 kB 10 %



using AI tools for analyzing data

- "can you write me SQL slicing clicking INP by release version?"
- "can you write SQL to tell me if the INP regression was more prominent for certain cohorts, such as plan type, browser, or OS?"
- "can you write SQL to tell me if the regression was driven by any particular click target?"

using github mcp for regressions

- "hey claude, we saw a regression in clicking INP occur in release 1786476972236. can you go through the PRs in that release and determine if anything could have caused that regression? please tell me why you think that could be the case. thanks!"

lessons
learned

performance issues are the
same they've always been.

AI has just intensified
them.

tried-and-true

engineering

best practices

remain the best

practices

AI tools should
supplement you,
not replace you!

thank you!

jenna.is/at-renderatl

@zeigenvector